

LLM API

Week 2 – Day 2 Assignment



JUNE 17, 2026

ANITHA PRAJISH

Contents

Objective:.....	3
Overview	3
API Name: Text from Image - Base64	3
Purpose:.....	3
Endpoint:	3
Headers Used:	3
Request Body:.....	3
Understanding of Request:	4
Example:	4
Response Received:	5
Understanding of Response:	5
Output:	5
API Name: Text to Audio	6
Purpose:.....	6
Endpoint:	6
HTTP Method:	6
Headers Used:	6
Request Body:.....	6
Understanding of Request:	6
Response Received:	7
Understanding of Response:	7
API Name: Multilingual.....	8
Purpose:.....	8
Endpoint:	8
HTTP Method:	8
Headers Used:	8
Request Body:.....	8
Understanding of Request:	8
Response Received:	9
Understanding of Response:	9
Output:	10
API Name: Generate API test cases from Swagger Code	11
Purpose:.....	11
Endpoint:	11
HTTP Method:	11
Headers Used:	11

Request Body:	11
Understanding of Request:	12
Response Received:	13
Understanding of Response:	13
COMMON OBSERVATIONS.....	14
KEY LEARNINGS	14
CONCLUSION	14

Objective:

To understand how Large Language Model (LLM) APIs work by sending API requests through Postman, analysing request structure, response behaviour, and understanding the purpose of each endpoint.

Overview

In this assignment, four APIs were provided for execution through Postman.

Tool Used: Postman

API Type: REST API

Data Format: JSON

Authentication Used: (API Key / Bearer Token / None)

API Name: Text from Image - Base64

Purpose:

This API is used to extract and retrieve text embedded within an image.

Endpoint:

<https://api.groq.com/openai/v1/chat/completions>

HTTP Method:

POST

Headers Used:

"Authorization": "<API key generated in Groq>"

"Postman-token": <calculated when request is sent>

"Content-Type": "application/json"

"Content-Length": <calculated when request is sent>

"Host": <calculated when request is sent>

"User-Agent": PostmanRunTime/7.54.0

"Accept": */*

"Accept-Encoding": gzip, deflate, br

"Connection": Keep-Alive

Request Body:

```
{  
  "model": "meta-llama/llama-4-scout-17b-16e-instruct",
```

```

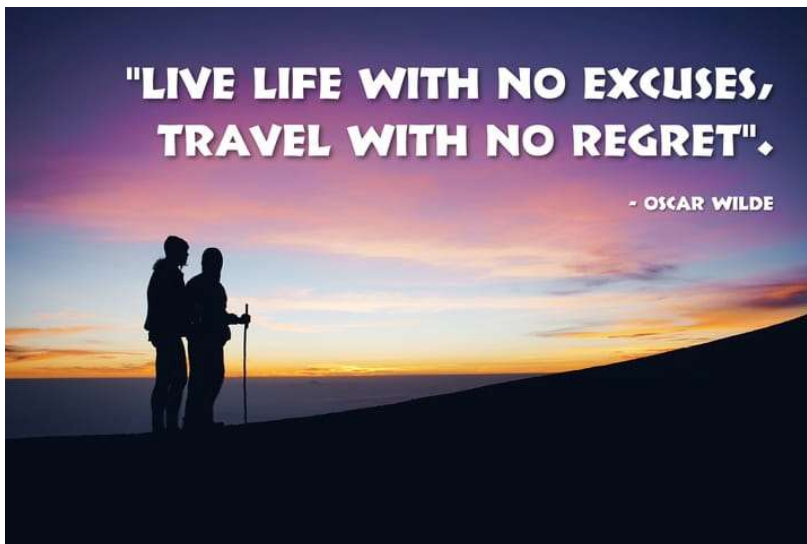
"messages": [
  {
    "role": "user",
    "content": [
      {
        "type": "text",
        "text": "This is a image. Extract and return only the text exactly as shown,
preserving case."
      },
      {
        "type": "image_url",
        "image_url": {
          "url": "data:image/png;base64,{{base64_img}}"
        }
      }
    ]
  }
]
}

```

Understanding of Request:

- This API is used to extract text from an image using an LLM model.
- The model parameter specifies which AI model processes the request.
- The messages field contains the input sent to the model.
- The text instruction tells the model to extract only the text exactly as shown in the image.
- The image_url field sends the image as Base64 encoded data.
- The API processes both text + image input (multimodal input).
- The model analyzes the image and identifies the text present in it.
- The response returns only the extracted text while preserving the original case

Example:



Response Received:

The screenshot displays a REST client interface with a sidebar on the left containing a list of collections and environments. The main area shows a POST request to `https://api.groq.com/openai/v1/chat/completions`. The request body is in raw JSON format, containing a text prompt and an image URL. The response is a 200 OK status with a JSON body. The response body is shown in a preview view, highlighting the generated text output.

```
POST https://api.groq.com/openai/v1/chat/completions

{
  "messages": [
    {
      "role": "user",
      "content": "This is a image. Extract and return only the text exactly as shown, preserving case."
    },
    {
      "role": "assistant",
      "content": "\nLIVE LIFE WITH NO EXCUSES, \nTRAVEL WITH NO REGRET\n\n- OSCAR WILDE"
    }
  ]
}

200 OK - 735 ms - 1.08 KB

{
  "id": "chatcmpl-08c6a6e2-d2e8-47e6-a299-c9529e11aab8",
  "object": "chat.completion",
  "created": 1781606578,
  "model": "meta-llama/llama-4-scout-17b-16e-instruct",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "\nLIVE LIFE WITH NO EXCUSES, \nTRAVEL WITH NO REGRET\n\n- OSCAR WILDE"
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ]
}
```

Understanding of Response:

- The id field provides a unique identifier for the API request.
- The object field indicates that the response is a chat completion generated by the model.
- The model field shows which AI model processed the request (Llama 4 Scout 17B Instruct).
- The choices field contains the actual response generated by the model.
- Inside message content, the API returns the text extracted from the image.
- The role field shows that the response was generated by the assistant.
- The finish_reason: stop indicates the response was completed successfully.
- The usage field shows token usage and processing time for the request.
- prompt_tokens represent input tokens sent to the model.
- completion_tokens represent output tokens generated in the response.
- total_tokens shows the total number of tokens used for processing the request

Output:

The API returned generated text output based on the input prompt.

"LIVE LIFE WITH NO EXCUSES,
TRAVEL WITH NO REGRET"
- OSCAR WILDE

API Name: Text to Audio

Purpose:

The purpose of this API is to convert text input into audio output.

Endpoint:

<https://api.groq.com/openai/v1/audio/speech>

HTTP Method:

POST

Headers Used:

"Authorization": "<API key generated in Groq>"
"Postman-token": <calculated when request is sent>
"Content-Type": "application/json"
"Content-Length": <calculated when request is sent>
"Host": <calculated when request is sent>
"User-Agent": PostmanRunTime/7.54.0
"Accept": */*
"Accept-Encoding": gzip, deflate, br
"Connection": Keep-Alive
"Authorization": Bearer \$Groq_API_Key

Request Body:

```
{
  "model": "canopylabs/orpheus-v1-english",
  "input": "[Cheerful] vanakkam. I am a language model created by Groq. I am here to help you with any questions you may have. How can I assist you today?",
  "voice": "diana",
  "response_format": "wav"
}
```

Understanding of Request:

- The API request is used to convert text input into speech/audio output (Text-to-Speech functionality).
- The model field specifies the AI model being used, which is canopylabs/orpheus-v1-english for English speech generation.
- The input field contains the text that needs to be converted into audio. In this case, it is a greeting message starting with "vanakkam..."
- The voice parameter defines the voice style/persona for the generated speech, here set as diana.
- The response_format parameter specifies the output audio format, which is WAV (wav).
- The API processes the provided text and generates an audio file in WAV format using the selected voice.

- The request demonstrates how AI can generate natural-sounding speech from text input for voice-based applications.

Response Received:

The screenshot displays a REST client interface with a sidebar on the left showing a collection of API requests under 'GenAI-June'. The main panel shows a POST request to `https://api.groq.com/openai/v1/audio/speech`. The request body is in JSON format, specifying the model as `canopylabs/orpheus-v1-english`, the input text as `"[Cheerful] vanakkam. I am a language model created by Groq. I am here to help you with any questions you may have. How can I assist you today?"`, the voice as `"diana"`, and the response format as `"wav"`. The response status is `200 OK` with a duration of `1.50 s` and a size of `402.36 KB`. Below the response details, there is a media player showing a black audio waveform, indicating that the response is an audio file.

Understanding of Response:

- The API returns the processed result after converting the given text input into speech.
- The output is generated as an audio file in the format specified in the request, which is WAV (.wav).
- The speech is created using the selected diana voice, producing natural-sounding audio.
- The input text message is converted into spoken audio while preserving the intended tone/style (in this case, cheerful).
- The generated output can be played, stored, or downloaded as an audio file.
- The successful output confirms that the Text-to-Speech conversion process was completed correctly.
- The response demonstrates the API's ability to transform text content into human-like speech output for audio-based applications.

API Name: Multilingual

Purpose:

This API is designed to produce output in the same language as the language used in the input.

Endpoint:

<https://api.groq.com/openai/v1/chat/completions>

HTTP Method:

POST

Headers Used:

"Authorization": "<API key generated in Groq>"
"Postman-token": <calculated when request is sent>
"Content-Type": "application/json"
"Content-Length": <calculated when request is sent>
"Host": <calculated when request is sent>
"User-Agent": PostmanRunTime/7.54.0
"Accept": */*
"Accept-Encoding": gzip, deflate, br
"Connection": Keep-Alive

Request Body:

```
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "You are an API Specialist. Always respond in the same language as the
user."
    },
    {
      "role": "user",
      "content": "Merci beaucoup ! Pourriez-vous m'expliquer comment utiliser cette API ?"
    }
  ]
}
```

Understanding of Request:

- The API request is used to interact with a Large Language Model (LLM) for generating text-based responses.

- The model field specifies the AI model being used, which is llama-3.3-70b-versatile for conversational text generation.
- The messages array provides the conversation context for the model.
- The system role defines instructions for the model, telling it to act as an API Specialist.
- The system message also instructs the model to always respond in the same language as the user's input.
- The user role contains the actual query, which in this case is written in French asking how to use the API.
- The API processes both system instructions and user input to generate a context-aware response.
- Since the user input is in French, the expected output will also be generated in French language.

Response Received:

The screenshot displays a REST client interface with a POST request to `https://api.groq.com/openai/v1/chat/completions`. The request body is a JSON object with a system message and a user message in French. The response is a 200 OK status with a JSON body containing the assistant's response in French.

Request Body (JSON):

```

{
  "messages": [
    {
      "role": "system",
      "content": "You are an API Specialist. Always respond in the same language as the user."
    },
    {
      "role": "user",
      "content": "Merci beaucoup ! Pourriez-vous m'expliquer comment utiliser cette API ?"
    }
  ]
}

```

Response Body (JSON):

```

{
  "index": 0,
  "message": {
    "role": "assistant",
    "content": "Bien sûr ! Je serais ravi de vous aider à comprendre comment utiliser cette API.\n\nTout d'abord, il est important de noter que les API (Application Programming Interface) sont des interfaces de programmation qui permettent à différentes applications de communiquer entre elles. Dans le cas de l'API que vous souhaitez utiliser, il est probable que vous deviez suivre les étapes suivantes :\n\n1. **Inscription et clé d'accès** : vous devrez peut-être vous inscrire pour obtenir une clé d'accès ou un jeton d'authentification qui vous permettra d'utiliser l'API.\n2. **Documentation de l'API** : lisez la documentation de l'API pour comprendre les endpoints disponibles, les méthodes HTTP autorisées (GET, POST, PUT, DELETE, etc.) et les paramètres requis pour chaque endpoint.\n3. **Choix de la méthode de requête** : sélectionnez la méthode de
  "
}

```

Understanding of Response:

- The API returns a successful text response generated by the llama-3.3-70b-versatile language model.
- The response is generated in French, matching the language of the user's input as instructed in the request.
- The model responds as an API Specialist, providing guidance related to API usage.
- The output begins by asking the user for more details about the specific API they want help with.
- The response provides general steps for using an API, such as checking documentation, obtaining API keys, making requests, and processing responses.

- The finish_reason: "stop" indicates that the response was completed successfully without interruption.
- The usage details show the number of tokens processed for both input and generated output.
- The response confirms that the API can generate context-aware, multilingual, and instruction-following outputs.

Output:

Bien sûr ! Je serais ravi de vous aider à comprendre comment utiliser cette API.

Tout d'abord, il est important de noter que les API (Application Programming Interface) sont des interfaces de programmation qui permettent à différentes applications de communiquer entre elles. Dans le cas de l'API que vous souhaitez utiliser, il est probable que vous deviez suivre les étapes suivantes :

1. ****Inscription et clé d'accès**** : vous devrez peut-être vous inscrire pour obtenir une clé d'accès ou un jeton d'authentification qui vous permettra d'utiliser l'API.
2. ****Documentation de l'API**** : lisez la documentation de l'API pour comprendre les endpoints disponibles, les méthodes HTTP autorisées (GET, POST, PUT, DELETE, etc.) et les paramètres requis pour chaque endpoint.
3. ****Choix de la méthode de requête**** : sélectionnez la méthode de requête appropriée pour votre besoin spécifique (par exemple, récupérer des données, créer une nouvelle ressource, mettre à jour une ressource existante, etc.).
4. ****Construction de la requête**** : construisez votre requête en incluant les paramètres requis, tels que les en-têtes, les données de formulaire, les paramètres de requête, etc.
5. ****Envoi de la requête**** : envoyez votre requête à l'endpoint de l'API en utilisant la méthode de requête choisie.
6. ****Traitement de la réponse**** : traitez la réponse reçue de l'API, qui peut inclure des données, des erreurs, des messages de succès, etc.

Pouvez-vous me donner plus de détails sur l'API que vous souhaitez utiliser et ce que vous essayez d'accomplir ? Je serais heureux de vous fournir des conseils plus spécifiques.

API Name: Generate API test cases from Swagger Code

Purpose:

The API helps in automating API testing by converting Swagger definitions into structured test cases for quality assurance.

Endpoint:

<https://api.groq.com/openai/v1/chat/completions>

HTTP Method:

POST

Headers Used:

"Authorization": "<API key generated in Groq>"

"Postman-token": <calculated when request is sent>

"Content-Type": "application/json"

"Content-Length": <calculated when request is sent>

"Host": <calculated when request is sent>

"User-Agent": PostmanRunTime/7.54.0

"Accept": */*

"Accept-Encoding": gzip, deflate, br

"Connection": Keep-Alive

Request Body:

```
{
  "model": "openai/gpt-oss-120b",
  "messages": [
    {
      "role": "system",
      "content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
    },
    {
      "role": "user",
      "content": "I — INSTRUCTION\nGenerate Rest Assured Java code for Swagger Petstore APIs.\n\nMANDATORY RULES:\n1) Generate ONLY Java Rest Assured code.\n2) Do NOT include explanations, markdown, or extra text.\n3) Use TestNG as the test runner.\n4) Use meaningful class, method, and variable names.\n5) Include imports required for Rest Assured, TestNG, and assertions.\n6) Use reusable base URI setup.\n7) Cover CRUD operations for Pet resource.\n8) Add assertions for status code and important response fields.\n9) Keep the code clean, executable, and production-style.\n10) Do NOT use placeholder comments.\n\nC — CONTEXT\nThe code will be used by QA engineers for API automation practice using Swagger Petstore.\n\nP — PERSONA\nYou are a Senior API Test
```

Automation Architect creating enterprise-style reusable Rest Assured automation code.\n\nO
 — OUTPUT\nReturn ONLY executable Java code with:\n- One test class\n- @BeforeClass
 setup\n- Test methods for createPet, getPetById, updatePet, deletePet\n- Request body
 creation\n- Response validation\n\nRules:\n- No markdown fences\n- No explanation text\n-
 No assumptions outside Swagger Petstore Pet API\n- Use business-readable method
 names\n\nTARGET API:\nSwagger Petstore\nResource: Pet\nOperations required:\n- Add a
 new pet\n- Find pet by ID\n- Update an existing pet\n- Delete a pet"

```

    }
  ]
}
```

Understanding of Request:

- The API request is used to interact with a Large Language Model (LLM) to generate automated API test code.
- The model field specifies the AI model being used, which is openai/gpt-oss-120b for advanced code generation.
- The system role defines the model's behaviour, instructing it to act as a Senior API Test Automation Architect.
- The request asks the model to generate Java automation code using Rest Assured framework.
- The generated code must use TestNG as the test runner and Maven-compatible imports.
- The request specifically targets Swagger Petstore API for testing the Pet resource.
- The code should cover complete CRUD operations: Create, Read, Update, and Delete for pet APIs.
- The request requires reusable base URI setup along with proper request body creation.
- The generated code must include response validations, such as status code checks and important field assertions.
- The output should be clean, executable, production-style Java code without explanations, markdown, or extra text.

Response Received:

The screenshot displays a REST client interface with a POST request to `https://api.groq.com/openai/v1/chat/completions`. The request body is a JSON object with the following structure:

```
1 {
2   "model": "openai/gpt-oss-120b",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are a Senior API Test Automation Architect with strong
7         expertise in Java, Rest Assured, TestNG, Maven, and API schema-driven test
8         generation."
9     }
10  ]
11 }
```

The response is a 200 OK status with a body containing a JSON object:

```
7 {
8   "index": 0,
9   "message": {
10     "role": "assistant",
11     "content": "import io.restassured.RestAssured;\nimport io.
    restassured.http.ContentType;\nimport io.restassured.response.
    Response;\nimport org.testng.Assert;\nimport org.testng.
    annotations.BeforeClass;\nimport org.testng.annotations.Test;
    \nimport java.util.HashMap;\nimport java.util.Map;\n\npublic
    class PetStoreApiCrudTest {\n\n    private static long petId;
    \n    private static final String BASE_URI = \"https://petstore.
    swagger.io/v2\";\n\n    @BeforeClass\n    public void setup()
    {\n        RestAssured.baseURI = BASE_URI;\n    }\n\n    @Test\n    public void createPet() {\n        petId = System.
    currentTimeMillis();\n        Map<String, Object> pet = new
    HashMap<>();\n        pet.put(\"id\", petId);\n        pet.put
    (\"name\", \"Rex\");\n        pet.put(\"status\", \"available\");
    }
    }
```

Understanding of Response:

- The API successfully generates Java automation test code based on the given Swagger Petstore API requirements.
- The response provides executable code using Rest Assured framework for API automation testing.
- The generated code uses TestNG as the test runner for managing test execution.
- A `@BeforeClass` setup method is included to configure the reusable base URI for API requests.
- The response covers complete CRUD operations for the Pet resource in Swagger Petstore.
- Separate test methods are created for Create Pet, Get Pet by ID, Update Pet, and Delete Pet operations.
- The code includes request body creation using Java objects/maps for sending API data.
- Each API request contains response validations, including status code checks and important field assertions.
- The response follows a clean, production-style coding structure with proper imports and meaningful method names.
- The successful output shows that the API can automatically generate ready-to-execute API automation scripts from API specifications.

COMMON OBSERVATIONS

While working with the APIs, I observed:

- APIs communicate using JSON format
- Authentication is required before making requests
- Request body parameters determine API behaviour
- Responses contain structured output data
- APIs process input and return model-generated output
- Different endpoints perform different tasks

KEY LEARNINGS

Through this assignment, I learned:

- Basics of API communication
- How Postman sends API requests
- Understanding HTTP methods
- Authentication handling in APIs
- Structure of JSON request/response
- How LLM APIs process user input and generate output

CONCLUSION

This assignment helped me understand how LLM APIs work internally.

By using Postman, I learned how requests are sent, how APIs process data, and how responses are generated.

It provided hands-on experience in understanding API communication and working with AI-powered endpoints.